

Module 10 Lab Session 1: Breaking the CORS Lock

Module	M10 Full-Stack Integration
Session	1 of 3
Exercises	Exercise 1 : Named CORS policy, environment driven API base URLs, CourseService wiring

Story Thread

Liya is testing the new TMS enrollment page. On her phone and browser, she clicks **Enroll in Web Architecture**. Nothing happens. No loading spinner, no error banner just silence.

She opens Chrome Developer Tools (F12), switches to the Console tab, and sees a bright red error message: Access to XMLHttpRequest at 'http://localhost:5000/api/v1/courses' from origin 'http://localhost:4200' has been blocked by CORS policy.

Yet, earlier that morning, Dawit tested the exact same endpoint using Scalar and Postman, and both received clean 200 OK responses.

Why did desktop API tools succeed while Chrome blocked Liya's browser?

Scalar and Postman are backend utilities; they do not enforce the browser's **Same-Origin Policy (SOP)**. Chrome, on the other hand, detects that your Angular application runs on localhost:4200 while your .NET API listens on localhost:5000. Because the port numbers differ, the browser treats them as two entirely separate websites and blocks JavaScript from reading the API's response.

In this lab, you will configure your .NET API to grant explicit permission to your Angular client and organize environment driven configuration URLs so your app is ready for production.

Exercise 1: Breaking the CORS Lock (Policy and Environments)

Part A: Diagnose the Lock in DevTools

Before changing a single line of code, let's observe the browser's security enforcement firsthand:

1. Open your TmsApi project and comment out `app.UseCors("AllowAngular");` registered by previous sessions;
2. Launch your .NET Web API project in one terminal window:

```
dotnet run
```
3. Launch your Angular application in a second terminal window:

```
ng serve
```
4. Open your browser to `http://localhost:4200` and open **DevTools** (F12 or right click -> Inspect).
5. Navigate to the Network tab and clear the log.
6. Trigger an HTTP request from Angular to your backend API.
7. Notice that the Network request appears red or fails to complete, and the Console displays the CORS blockage message.

Notice something subtle: the server actually processed the request and returned a response! But Chrome intercepted that response before your Angular TypeScript code could touch it.

Part B: Configure a Named CORS Policy in .NET 10

To tell Chrome that our Angular frontend is a trusted partner, we must declare a CORS policy on the .NET server.

1. Open `appsettings.Development.json` in your Web API project. Add the allowed origin list so we do not hardcode URLs in C# source code:

```
{  
  "AllowedOrigins": [  
    "http://localhost:4200"  
  ]  
}
```

2. Open `Program.cs` in your Web API project.

3. Locate the service registration section (before `builder.Build()`) and define a named CORS policy called "TmsClient":

```
// Load allowed origins from appsettings.Development.json
var allowedOrigins = builder.Configuration
    .GetSection("AllowedOrigins").Get<string[]>()
    ?? ["http://localhost:4200"];

// Register the CORS policy in the Dependency Injection container
builder.Services.AddCors(options =>
{
    options.AddPolicy("TmsClient", policy =>
    {
        policy.WithOrigins(allowedOrigins)
            .AllowAnyHeader()
            .AllowAnyMethod()
            .AllowCredentials() // Vital for HttpOnly auth cookies in Session 2
            .SetPreflightMaxAge(TimeSpan.FromMinutes(10));
    });
});
```

4. Scroll down to the HTTP request pipeline configuration section (after `builder.Build()`). Enable the policy using `app.UseCors()`:

```
// CRITICAL: Middleware order matters!
// UseRouting -> UseCors -> UseAuthentication -> UseAuthorization
app.UseCors("TmsClient");
```

A Crucial Security Trap to Avoid: Never combine `.AllowAnyOrigin()` with `.AllowCredentials()`. If you attempt to do so, ASP.NET Core will throw an `InvalidOperationException` at server startup. The browser specification strictly forbids wildcard origins when sending authenticated credentials like cookies or auth headers, because doing so would allow any malicious site on the web to make credentialed calls against your user's session.

5. Save `Program.cs`.
6. Stop your API terminal (Ctrl+C) and restart it (`dotnet run`). Configuration changes in `Program.cs` require a full process restart to take effect.

Part C: Clean Environment Configurations and Domain Models

Hardcoding URLs like `http://localhost:5000` inside your services makes deploying to production painful. Let's create Angular environment configurations to handle API base routes cleanly.

1. Angular 22 does not generate environment files by default. Run the generator in your Angular project terminal:

```
ng generate environments
```

This command creates `src/environments/environment.ts` and `src/environments/environment.development.ts`, while updating `angular.json` automatically.

2. Open `src/environments/environment.development.ts` and set your local API endpoint:

```
export const environment = {  
  production: false,  
  apiUrl: '/api/v1'  
};
```

3. Open `src/environments/environment.ts` (used for production builds):

```
export const environment = {  
  production: true,  
  apiUrl: '/api/v1'  
};
```

4. Verify your Course interface and PagedResponse<T> wrapper in `src/app/models/course.model.ts`:

```
export interface Course {  
  id: number;  
  code: string;  
  title: string;  
  maxCapacity: number;  
  enrollmentCount: number;  
  status?: string;  
}  
  
export interface PagedResponse<T> {  
  items: T[];  
  totalCount: number;  
  page: number;  
  pageSize: number;  
  totalPages: number;
```

```

    hasPrevious: boolean;
    hasNext: boolean;
  }

```

5. Update your CourseService (src/app/services/course.service.ts) to use the environment config and modern Angular @Service() injection patterns:

```

import { Service, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { map } from 'rxjs/operators';
import { environment } from '../../environments/environment';
import { Course, PagedResponse } from '../../models/course.model';

@Service()
export class CourseService {
  private http = inject(HttpClient);
  private readonly base = `${environment.apiUrl}/courses`;

  getAll() {
    return this.http
      .get<PagedResponse<Course>>(this.base, {
        params: { page: '1', pageSize: '50' }
      })
      .pipe(map(response => response.items));
  }
}

```

Verifying Your Work

Switch back to your browser at <http://localhost:4200> and open DevTools to inspect your request:

1. **Successful Communication:** The network request returns HTTP status 200 OK.
2. **Clean Console:** The browser console displays no red CORS error messages.
3. **Header Verification:** Click a network request to your API. In the Response Headers you will see Access-Control-Allow-Origin:
<http://localhost:4200> proof that the named CORS policy is working.

What's Next?

With the CORS lock broken and API communication established, your Angular app can read data from your backend. But what happens when Liya tries to perform an action that requires logging in?